

Exercise1:

```
class SingletonPatternExample{

    static class Logger{

        private static Logger object;

        private String name;

        private Logger(String name){

            this.name = name;

        }

        public static Logger get(){

            if(object==null){

                object = new Logger("something");

            }

            return object;

        }

    }

    public static void main (String args[]){

        Logger g1 = Logger.get();

        Logger g2 = Logger.get();

        if(g1 == g2){

            System.out.println("same object");

        }else{

            System.out.println("different objects");

        }

    }

}
```

Exercise2:

```
public class FactoryPatternExample {  
    interface document {  
        void display();  
    }  
  
    static class WordDocument implements document {  
        public void display() {  
            System.out.println("word");  
        }  
    }  
  
    static class PdfDocument implements document {  
        public void display() {  
            System.out.println("pdf");  
        }  
    }  
  
    static class ExcelDocument implements document {  
        public void display() {  
            System.out.println("excel");  
        }  
    }  
  
    static abstract class DocumentFactory {  
        abstract document createDocument();  
    }  
  
    static class WordFactory extends DocumentFactory {  
        public document createDocument() {  
            return new WordDocument();  
        }  
    }  
}
```

```

    }
}

static class PdfFactory extends DocumentFactory {

    public document createDocument() {

        return new PdfDocument();

    }

}

static class ExcelFactory extends DocumentFactory {

    public document createDocument() {

        return new ExcelDocument();

    }

}

public static void main(String args[]) {

    DocumentFactory factory;

    factory = new PdfFactory();

    document d = factory.createDocument();

    d.display();

    factory = new WordFactory();

    d = factory.createDocument();

    d.display();

}

}

```

Exercise3:

```

class BuilderPatternExample{

    static class Computer{

        String cpu;

```

```
String ram;

String storage;

Computer(Builder builder){

    this.cpu = builder.cpu;

    this.ram = builder.ram;

    this.storage = builder.storage;

}

void display() {

    System.out.println(cpu);

    System.out.println(ram);

    System.out.println(storage);

}

}

static class Builder{

    String cpu;

    String ram;

    String storage;


    public Builder setcpu(String cpu){

        this.cpu = cpu;

        return this;

    }

    public Builder setram(String ram){

        this.ram = ram;

        return this;

    }

    public Builder setstorage(String storage){

        this.storage= storage;
```

```

        return this;
    }

    public Computer build(){
        return new Computer(this);
    }
}

public static void main(String args[]){
    Computer c = new Builder().setcpu("A").setram("B").setstorage("C").build();
    c.display();
}
}

```

Exercise 4:

```

interface PaymentProcessor {
    void processPayment(double amount);
}

class PayPal {
    void makePayment(double amount) {
        System.out.println("PayPal payment successful: ₹" + amount);
    }
}

class Stripe {
    void payAmount(double amount) {
        System.out.println("Stripe payment successful: ₹" + amount);
    }
}

class PayPalAdapter implements PaymentProcessor {

```

```

PayPal paypal;

PayPalAdapter(PayPal paypal) {
    this.paypal = paypal;
}

public void processPayment(double amount) {
    paypal.makePayment(amount);
}
}

class StripeAdapter implements PaymentProcessor {

    Stripe stripe;

    StripeAdapter(Stripe stripe) {
        this.stripe = stripe;
    }

    public void processPayment(double amount) {
        stripe.payAmount(amount);
    }
}

public class AdapterPatternExample {
    public static void main(String[] args) {
        PaymentProcessor payment;

        payment = new PayPalAdapter(new PayPal());
        payment.processPayment(500);

        payment = new StripeAdapter(new Stripe());
        payment.processPayment(1000);
    }
}

```

```
}
```

Exercise 5:

```
interface Notifier {  
    void send(String message);  
}  
  
class EmailNotifier implements Notifier {  
    public void send(String message) {  
        System.out.println("Email notification: " + message);  
    }  
}  
  
abstract class NotifierDecorator implements Notifier {  
    protected Notifier notifier;  
    NotifierDecorator(Notifier notifier) {  
        this.notifier = notifier;  
    }  
    public void send(String message) {  
        notifier.send(message);  
    }  
}  
  
class SMSNotifierDecorator extends NotifierDecorator {  
    SMSNotifierDecorator(Notifier notifier) {  
        super(notifier);  
    }  
    public void send(String message) {  
        super.send(message);  
        System.out.println("SMS notification: " + message);  
    }  
}
```

```

}

class SlackNotifierDecorator extends NotifierDecorator {

    SlackNotifierDecorator(Notifier notifier) {

        super(notifier);

    }

    public void send(String message) {

        super.send(message);

        System.out.println("Slack notification: " + message);

    }

}

public class DecoratorPatternExample {

    public static void main(String[] args) {

        Notifier notifier = new EmailNotifier();

        notifier = new SMSNotifierDecorator(notifier);

        notifier = new SlackNotifierDecorator(notifier);

        notifier.send("Your order has been placed successfully");

    }

}

```

Exercise 6:

```

interface Image {

    void display();

}

class ReallImage implements Image {

    private String fileName;

    ReallImage(String fileName) {

        this.fileName = fileName;

        loadImage();

    }

}

```



```

    }

    void loadImage() {
        System.out.println("Loading image from remote server: " + fileName);
    }

    public void display() {
        System.out.println("Displaying image: " + fileName);
    }
}

class ProxyImage implements Image {
    private String fileName;
    private ReallImage reallImage;

    ProxyImage(String fileName) {
        this.fileName = fileName;
    }

    public void display() {
        if (reallImage == null) {
            reallImage = new ReallImage(fileName);
        }

        reallImage.display();
    }
}

public class ProxyPatternExample {
    public static void main(String[] args) {
        Image image = new ProxyImage("photo.jpg");
        image.display();

        System.out.println("Calling again:");
        image.display();
    }
}

```

```
}  
}
```

Exercise 7:

```
import java.util.ArrayList;  
import java.util.List;  
  
interface Stock {  
    void registerObserver(Observer observer);  
    void removeObserver(Observer observer);  
    void notifyObservers();  
}  
  
interface Observer {  
    void update(double price);  
}  
  
class StockMarket implements Stock {  
    private List<Observer> observers = new ArrayList<>();  
    private double price;  
    public void registerObserver(Observer observer) {  
        observers.add(observer);  
    }  
    public void removeObserver(Observer observer) {  
        observers.remove(observer);  
    }  
    public void setPrice(double price) {  
        this.price = price;  
        notifyObservers();  
    }  
}
```

```

    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(price);
        }
    }
}

class MobileApp implements Observer {
    public void update(double price) {
        System.out.println("Mobile App: Stock price changed to ₹" + price);
    }
}

class WebApp implements Observer {
    public void update(double price) {
        System.out.println("Web App: Stock price changed to ₹" + price);
    }
}

public class ObserverPatternExample {
    public static void main(String[] args) {
        StockMarket market = new StockMarket();

        Observer mobile = new MobileApp();
        Observer web = new WebApp();

        market.registerObserver(mobile);
        market.registerObserver(web);

        market.setPrice(1500);
        market.setPrice(1600);
        market.removeObserver(web);
        market.setPrice(1700);
    }
}

```

```
}
```

Exercise 8:

```
interface PaymentStrategy {
```

```
    void pay(double amount);
```

```
}
```

```
class CreditCardPayment implements PaymentStrategy {
```

```
    public void pay(double amount) {
```

```
        System.out.println("Paid ₹" + amount + " using Credit Card");
```

```
    }
```

```
}
```

```
class PayPalPayment implements PaymentStrategy {
```

```
    public void pay(double amount) {
```

```
        System.out.println("Paid ₹" + amount + " using PayPal");
```

```
    }
```

```
}
```

```
class PaymentContext {
```

```
    private PaymentStrategy strategy;
```

```
    PaymentContext(PaymentStrategy strategy) {
```

```
        this.strategy = strategy;
```

```
    }
```

```
    void executePayment(double amount) {
```

```
        strategy.pay(amount);
```

```
    }
```

```
}
```

```
public class StrategyPatternExample {
```

```
    public static void main(String[] args) {
```

```
    PaymentContext payment;

    payment = new PaymentContext(new CreditCardPayment());

    payment.executePayment(500);

    payment = new PaymentContext(new PayPalPayment());

    payment.executePayment(1000);

}

}
```

Exercise 9:

```
interface Command {

    void execute();

}

class Light {

    void turnOn() {

        System.out.println("Light is ON");

    }

    void turnOff() {

        System.out.println("Light is OFF");

    }

}

class LightOnCommand implements Command {

    Light light;

    LightOnCommand(Light light) {

        this.light = light;

    }

    public void execute() {

        light.turnOn();

    }

}
```

```

}

class LightOffCommand implements Command {
    Light light;

    LightOffCommand(Light light) {
        this.light = light;
    }

    public void execute() {
        light.turnOff();
    }
}

class RemoteControl {
    Command command;

    void setCommand(Command command) {
        this.command = command;
    }

    void pressButton() {
        command.execute();
    }
}

public class CommandPatternExample {
    public static void main(String[] args) {
        Light light = new Light();

        RemoteControl remote = new RemoteControl();

        remote.setCommand(new LightOnCommand(light));

        remote.pressButton();

        remote.setCommand(new LightOffCommand(light));

        remote.pressButton();
    }
}

```

```
}
```

Exercise 10:

```
class Student {  
    private String name;  
    private int id;  
    private String grade;  
    Student(String name, int id, String grade) {  
        this.name = name;  
        this.id = id;  
        this.grade = grade;  
    }  
    String getName() {  
        return name;  
    }  
    int getId() {  
        return id;  
    }  
    String getGrade() {  
        return grade;  
    }  
    void setGrade(String grade) {  
        this.grade = grade;  
    }  
}  
  
class StudentView {  
    void displayStudentDetails(String name, int id, String grade) {  
        System.out.println("Student Name: " + name);  
    }  
}
```

```

        System.out.println("Student ID: " + id);

        System.out.println("Student Grade: " + grade);
    }
}

class StudentController {
    Student student;

    StudentView view;

    StudentController(Student student, StudentView view) {
        this.student = student;
        this.view = view;
    }

    void updateGrade(String grade) {
        student.setGrade(grade);
    }

    void display() {
        view.displayStudentDetails(
            student.getName(),
            student.getId(),
            student.getGrade()
        );
    }
}

public class MVCPatternExample {
    public static void main(String[] args) {
        Student student = new Student("Pranavi", 101, "A");

        StudentView view = new StudentView();

        StudentController controller =
            new StudentController(student, view);
    }
}

```



```

        controller.display();

        controller.updateGrade("A+");

        controller.display();
    }
}

```

Exercise 11:

```

interface CustomerRepository {

    String findCustomerById(int id);

}

class CustomerRepositoryImpl implements CustomerRepository {

    public String findCustomerById(int id) {

        return "Customer found with ID: " + id;

    }

}

class CustomerService {

    private CustomerRepository repository;

    CustomerService(CustomerRepository repository) {

        this.repository = repository;

    }

    void getCustomer(int id) {

        String customer = repository.findCustomerById(id);

        System.out.println(customer);

    }

}

public class DependencyInjectionExample {

    public static void main(String[] args) {

```

```
CustomerRepository repository = new CustomerRepositoryImpl();  
CustomerService service = new CustomerService(repository);  
service.getCustomer(101);  
}  
}
```